Journal of Intelligent & Robotic Systems (2025) 111:3
https://doi.org/10.1007/s10846-024-02205-0

REGULAR PAPER

Path Planning for the Robotic Manipulator in Dynamic Environments Based on a Deep Reinforcement Learning Method

Jie Liu[1,2] · **Hwa Jen Yap**[1] · **Anis Salwa Mohd Khairuddin**[3]

Received: 24 January 2024 / Accepted: 15 November 2024

Abstract
Collaborative and autonomous robots are increasingly important in meeting the demands of a faster and more cost-effective market. To ensure production efficiency and safety, robots must swiftly respond to the presence of human operators or other dynamic obstacles, avoiding potential collisions by quickly planning alternative paths. Deep Reinforcement Learning (DRL) based methods have shown great potential in path planning due to their rapid response capabilities. However, existing DRL-based planners lack a safety verification system to evaluate the feasibility of actions generated by neural models, and they cannot guarantee 100% collision-free paths. This paper presents an enhanced DRL-based path planning system incorporating a robust safety verification mechanism. This system predicts potential collisions and generates alternative collision-free paths as necessary. We analyzed the essential elements of trajectory planning using the DRL method and proposed improvements to accelerate planning speed. The results demonstrate that our planner consistently generates paths for typical reaching tasks with an average planning time of 12.1 ms, a notable improvement over traditional algorithms. Moreover, the paths produced by our method are nearly optimal, akin to those generated by Optimization-based algorithms.

Keywords Path planning · Manipulator · Dynamic environment · Twin delayed DDPG (TD3) · Deep reinforcement learning

1 Introduction

An increasing demand for collaborative robotic manipulators arises in various industries, including healthcare, manufacturing, agriculture, firefighting, and security [1]. Recent advancements in sensing technology, motion planning, and computing technology enable manipulators to operate effectively in highly uncertain and dynamic environments [2, 3]. Introducing robots into tasks previously undertaken only by humans contributes to higher efficiency and quality.

Traditional robots often lack the perception and intelligence to respond dynamically to environmental changes, resulting in work stoppages when obstacles enter their workspace. This hinders human–robot collaboration and reduces overall efficiency. A more promising solution involves adjusting the manipulator's trajectory in real time to avoid potential collisions while continuing the primary task uninterrupted.

1.1 Sampling-Based Algorithms

As computing complexity increases exponentially with the dimensionality of the space, Graph search algorithms, such as Dijkstra and A*, become impractical for motion planning problems of multiple degrees of freedom (DOF) manipulators. Sampling-based algorithms [4–10] search for a valid path by randomly sampling the configuration space. The random feature can maintain high efficiency even for complex problems. Heuristic sampling methods further accelerate planning by biasing samples toward promising configurations. Bias-goal sampling method [6, 11–13] reduces redundant exploration by incorporating steps toward the goal configuration, enabling faster convergence. Experience-bias sampling methods sample

✉ Hwa Jen Yap
hjyap737@um.edu.my

1 Department of Mechanical Engineering, Faculty of Engineering, Universiti Malaya, 50603 Kuala Lumpur, Malaysia

2 Department of Electrical Engineering, Hebei Vocational University of Technology and Engineering, Xingtai 054000, China

3 Department of Electrical Engineering, Faculty of Engineering, Universiti Malaya, 50603 Kuala Lumpur, Malaysia

Published online: 21 December 2024

more frequently in promising regions, learning the sampling distribution from successful planning paths or demonstrations via neural networks [14–18]. Combination with other methods, such as Artificial Potential Field (APF), leads to higher replanning effectiveness [19, 20]. However, paths generated by sampling-based algorithms often lack smoothness, resulting in jerky motions. While post-processing techniques can improve path quality, their effectiveness diminishes in cluttered environments. Additionally, the inherent randomness of these methods can lead to variability in solutions across different runs.

1.2 Optimization-Based Algorithms

Optimization-based algorithms tackle motion planning by minimizing an objective function within several constraints. One prominent gradient-based method, Covariant Hamiltonian Optimization for Motion Planning (CHOMP) [21], uses covariant gradient descent to minimize both trajectory smoothness and obstacle costs, facilitating effective obstacle avoidance. Conversely, stochastic optimization methods such as Stochastic Trajectory Optimization for Motion Planning (STOMP) [17, 22], update trajectories using stochastic gradient estimates from noise trajectories, which helps to overcome local minima and enhance planning efficiency. Incremental Trajectory Optimization Motion Planning (ITOMP) [23] interleaves planning with task execution based on STOMP [22], enabling quick responses to dynamic environments. Incremental Gaussian Process Motion Planner 2 (GPMP2) [24, 25] can update obstacle information in real-time and replan optimal paths. Utilizing GPU-Voxels accelerates environment reconstruction, allowing real-time generation of collision-free trajectories using live sensor data [26]. Interleaved Sampling and Interior Point Optimization Motion Planning (ISIMP) [27] and JoInt Sampling and Trajectory optimization over graphs (JIST) [28] leverage the complementary strengths of exploration from sampling methods and exploitation from optimization methods. These approaches outperform both optimization-based and sampling-based methods in success rates, incurring only a slightly higher computation time than optimization-based methods, and significantly less than sampling-based methods. However, many optimization-based methods require an initial trajectory, which can limit their effectiveness if the initial guess is poor. Moreover, the computational demands of these methods can be high, especially in complex environments or tasks.

1.3 Deep Reinforcement Learning-Based Algorithms

Recently, the Deep Reinforcement Learning (DRL) method has shown great potential for path planning. DRL allows agents to interact with the environment and learn control policies for guiding manipulators in Markov decision processes (MDP). In the context of MDP, Q-learning [29, 30] traditionally stores optimal action values in a Q table, which cannot handle complex systems. Deep Q Network (DQN) [31–34] replaces the Q-table with a neural network, allowing it to handle large state spaces but is still primarily suited for discrete action spaces. Deep Deterministic Policy Gradient (DDPG) [35] extends DQN to continuous action spaces. By leveraging techniques such as experience replay and Critic Networks, DDPG mitigates instability issues. Its efficacy has been validated in dynamic reaching tasks [36–40].

Twin Delayed DDPG (TD3) [41] addresses overestimation bias in DDPG agents with three key modifications: maintaining two independent critic networks, delaying weight updates, and adding clipped noise to target actions. These adjustments have been validated in human–robot cooperation scenarios, enhancing learning stability and policy optimality [2, 42]. Soft Actor-Critic (SAC) improves stability in unstable environments by balancing expected return and entropy, promoting exploration and robustness simultaneously. SAC excels in autonomous obstacle avoidance planning, even in challenging dynamic environments, outperforming other algorithms in success rate, stability, and task completion time [43, 44].

To make better use of the training experiences, Prioritized Experience Replay (PER) [36, 45] prioritizes high-value samples and replays experiences with greater value more frequently. Hindsight Experience Replay (HER) [46, 47] not only learns from successful experiences but also from failures. By improving sample utilization, HER enables training even with sparse and binary reward signals, demonstrating superior performance in multi-goal problems. When integrated with algorithms like TD3 [48]or SAC [37, 49], HER-equipped planners can effectively handle dynamic obstacles. Hindsight Goal Generation (HGG) [50] outperforms HER in challenging tasks where goals are difficult to explore, learning from a curriculum. Graph-based HGG (G-HGG) [51] enhances HGG by selecting intermediate goals from a precomputed graph representation of the environment, though this is not valid in static scenarios. Bing et al. [52] extend G-HGG to bounding-box-based HGG in robot manipulation, which helps select hindsight goals according to environment image, making HGG capable of handling dynamic obstacles.

1.4 Challenges and Gaps in Existing Methods

Unlike traditional planning algorithms, DRL methods need to generate actions in real time based on the current state of the environment. Although the policy network can generate actions rapidly, the feasibility of these actions is not evaluated. The success rates of existing DRL-based methods are shown in Table 1.

In the area of autopilot, safety verification is essential during the operation of the vehicle [53], which is absent in DRL-based

Table 1 Success rates of existing DRL-based methods

Method	Task	Obstacle	Success Rate
[30]	Two robot arms in Reaching task	static	94.0%
[32]	Reaching task	static	77.5%
[38]	Reaching task	static	86.0%
[42]	Reaching task in bookshelf	static	98.0%
[43]	Obstacle avoidance while maintaining a trajectory	dynamic	91.4%
[44]	Obstacle avoidance in reaching task	dynamic	96.0%
[49]	Multi-arm manipulators with periodically moving obstacles	dynamic	81.8%
[52]	Fetching moving obstacle	dynamic	90.0%

path planning methods for manipulators. The absence of such verification can result in collisions during real-world operations. To address this, we develop an enhanced DRL-based planning system that incorporates a robust safety verification mechanism. This system predicts potential collisions and generates alternative collision-free paths as necessary.

Collision detection for robotic arms presents unique challenges compared to autonomous vehicles. While vehicle collision detection can often rely on direct computational methods, robotic arms require simulating their movements to specific trajectory points and performing geometry-based collision checks with obstacles [54–59]. This leads to longer collision detection times compared to numerical methods, making it all the more essential to shorten the overall planning time.

1.5 Proposed Planning Method

In our proposed system, the initial trajectory is computed based on the current positions of obstacles. During the execution of this trajectory by the robotic arm, the verification system operates in parallel, conducting real-time monitoring for potential collisions along the future trajectory in a simulated environment, as illustrated in Fig. 2. If a collision is detected, the planner dynamically generates and implements a new, feasible trajectory. This adaptive mechanism ensures the robotic arm maintains an uninterrupted and safe path, thereby enhancing reliability and safety in dynamic environments.

In generating the new trajectory, we analyzed the elements involved in the process and proposed three improvements to accelerate the planning. Firstly, instead of modifying the entire trajectory from scratch, we retain the collision-free portion of the current path and use a replanning module to reconnect the path nodes closest to the obstacle. Secondly, we implement a deterministic policy method, namely TD3, to generate actions more rapidly. Additionally, we simplify the state transition function by eliminating redundant calculations. These modifications result in a significant reduction in planning time and allow the path to be dynamically adjusted based on obstacle movements. To accelerate the training process of the DRL planner, we designed a reward function based on the potential field function in APF [60], which helps the agent learn to reach the target position while avoiding collisions with moving obstacles. We also compared our method with an HER-equipped planner to decide whether to integrate HER into our algorithm. Simulation experiments demonstrate that the proposed planning method can complete path replanning in an average of 12.1ms for a reaching task, significantly outperforming traditional algorithms such as Rapid-exploration Random Tree (RRT) [4] (45.3ms), Probabilistic Roadmaps (PRM) [61] (1410ms), and generate nearly optimal path as Optimization-based algorithms.

The rest of the paper is organized as follows: Section 2 formulates the planning problem, and Section 3 presents the design of the TD3 motion planner. Section 4 describes the framework for generating real-time trajectories and introduces the optimization method to improve planning efficiency. Section 5 details the simulation experiments and their results. Finally, Section 6 provides the conclusion and outlines future research directions.

2 Problem Formulation

For the problem of real-time path planning in a dynamic environment, we want to find a dynamic path, i.e. $(x_0, x_1, \ldots, x_{goal})$, which connects the state position x_0 with goal position x_{goal} and can be modified under the influence of the dynamic obstacle to avoid the collision. To solve this problem, we can first plan an original end-effector path with an offline path planner. As the agent approaches the goal position, once the obstacle moves in the middle of the path, the original path is modified in real-time to avoid potential collision. As the obstacle moves randomly, the new trajectory is only guaranteed to be safe within a short time interval. This means the time for path replanning is limited and the replanning process must be fast enough.

So, the problem can be described as follows: Given the current position of the end-effector, the target position, and an original path $(x_0, x_1, \ldots, x_{goal})$, we need to compute a new path $(x_0', x_1', \ldots, x_{goal}')$ in response to the movement of the obstacle in a limited amount of time.

3 Method

3.1 TD3 Motion Planner

Motion planning of multi-DOF manipulator can be described as a continuous state-action model in a high-dimensional space, DRL methods such as DDPG, TD3, and SAC can solve continuous action space tasks. While DDPG agents can overestimate the value function in the training process, leading to suboptimal policies. TD3 overcomes the overestimation of the value function. The network architecture of TD3 is shown in Fig. 1. Though SAC has been proven better performance in solving motion planning problems of manipulators, SAC spends more time in path planning as it needs to generate the actions from a policy distribution. TD3 spends less time as it generates actions deterministically.

TD3 is based on the actor-critic network architecture. There are 6 neural networks, including an actor network, an actor target network, two critic networks, and two critic target networks. The target networks are time-delayed copies of their original networks that slowly track the learned networks. The utilization of these target networks improves learning stability. The two critic networks are independent and the minimum value function of them is adopted as an optimization target to suppress the overestimation bias. The clipped normal noise added to the target action also makes the policy more likely to choose actions with low Q-value estimates.

In training the agent, the actor network receives state input describing the state of the environment, s_t, including information such as the angle of each joint, the position of the target position, and the obstacle position. Then it outputs the action of the manipulator, a_t, such as the angular velocity of each joint. Then, according to the distance between the current position and the target position of the end-effector, the environment returns an immediate reward, r_t. By constantly interacting with the environment and performing the appropriate actions, the manipulator can get to the target position. There are several termination conditions: (1) the end-effector of the manipulator reaches the target point, (2) the end-effector encounters an obstacle, (3) the end-effector exits the workspace, and (4) the number of steps interacting with the environment reaches the upper limit. The path-planning algorithm for the manipulator is in Algorithm 1.

Algorithm 1 Path-planning algorithm for the manipulator

1. Initialize the critic network with $Q_{\theta_1}, Q_{\theta_2}$ and the actor network with π_ϕ.
2. Initialize target networks:
$$\theta'_1 \leftarrow \theta_1,\ \theta'_2 \leftarrow \theta_2,\ \phi' \leftarrow \phi$$
3. Initialize replay buffer B.
4. **while** timesteps < total_timesteps **do**
5. Receive initial state s_1.
6. **for** t=1 to T **do**
7. a. Select the action of the manipulator with exploration noise:
$$a_t = \mu(s_t \mid \theta^\mu) + \varepsilon,\quad \varepsilon \sim N(0,\sigma).$$
8. b. The manipulator executes action a_t and observes reward r_t and new state s_{t+1}.
9. c. Store transition (s_t, a_t, r_t, s_{t+1}) in B.
10. d. Sample a random minibatch of N transitions (s_t, a_t, r_t, s_{t+1}) from B, and update the actor and the critic networks with minimum Q value:
$$\tilde{a} \leftarrow \pi_{\phi'}(s_{t+1}) + \varepsilon,\ \varepsilon \sim \mathrm{clip}(N(0,\tilde{\sigma}), -c, c)$$
$$y = r_t + \gamma \min Q_{\theta'_i}(s_{t+1}, \tilde{a})$$
$$\theta_i \leftarrow \arg\min_{\theta_i} N^{-1} \Sigma (y - Q_{\theta_i}(s,a))^2, i = 1,2$$
11. e. **if** t mod d **then**
12. update the target networks at every d step:
$$\theta'_i \leftarrow \tau\theta_i + (1-\tau)\,\theta'_i, i = 1,2$$
$$\phi' \leftarrow \tau\phi + (1-\tau)\phi'$$
13. **end if**
14. f. **if** terminated=True **then**
15. break
16. **end if**
17. **end for**
18. **end while**

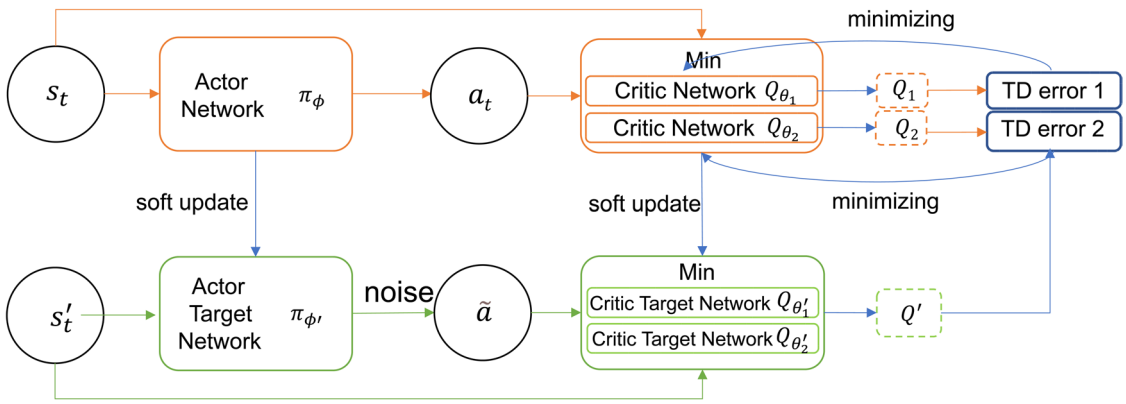


Fig. 1 Framework of TD3

3.2 Design of State Space and Action Space

The state s_t usually includes the angle of each manipulator's joint, but here we represent the state of the robot with the end-effector position $\{x_e, y_e, z_e\}$ in Euclidean space. This approach not only reduces the dimensionality of the state space, thereby shrinking the search space and improving the algorithm's convergence speed, but also makes it easier for the robot to approach the target due to the intuitive spatial representation of the end-effector position compared to joint angles. The state s_t also includes the position of the obstacle $\{x_o, y_o, z_o\}$ and the target object $\{x_t, y_t, z_t\}$. This allows the agent to gain a clear understanding of the robot's environment, facilitating both obstacle avoidance and movement toward the target. The full state description is:

$$s_t = \{x_e, y_e, z_e, x_o, y_o, z_o, x_t, y_t, z_t\}. \tag{1}$$

Correspondingly, the action space includes the displacement speed of the end effector in Euclidean space, which can be represented as

$$a_t = \{v_x, v_y, v_z\}. \tag{2}$$

Choosing velocity as the control variable offers more direct and intuitive motion planning, allowing for smoother and continuous control of the robot's trajectory.

After we get the new end-effector position, the joint angle of each joint is calculated through inverse kinematics calculation.

3.3 Design of Reward Function

DRL finds optimal policy through maximum cumulative reward. So, the design of the reward function directly affects learning efficiency and performance. Here we design a comprehensive dense reward function referring to the potential field function of APF.

3.3.1 Attractive Reward

To attract the agent to move toward the target position, the planner should return a positive reward as the manipulator approaches the target object. Here we propose this attractive reward as a function of the Euclidean distance d_t between the end-effector and the target.

$$d_t = \sqrt{(x_e - x_t)^2 + (y_e - y_t)^2 + (z_e - z_t)^2} \tag{3}$$

Once d_t is less than a predefined threshold $d_{th,}$ the end-effector is assumed to reach the target point. Then a huge positive reward, R_t, will be returned as a bonus reward. And this attractive reward can be represented as

$$r_a = \begin{cases} -\omega_1 d_t, d_t \geq d_{th} \\ R_t, d_t < d_{th} \end{cases}, \tag{4}$$

where ω_1 is the weighted factor.

3.3.2 Repulsive Reward

Realization of obstacle avoidance requires a collision penalty reward r_c. Once the robot gets into the influence area of the obstacle, a repulsive reward will be generated to push the links away. And upon collision, the environment returns a huge negative reward R_c to penalty collision. Suppose there are k pairs of points whose distances d_c are less than the influence distance d_{in}, the repulsive reward r_c can be defined as follows.

$$r_c = \begin{cases} \omega_2 \sum_{i=1}^{k} \left(d_c(i) - d_{in}\right), d_c(i) < d_{in} \\ -R_c, \text{collision=true} \end{cases}. \tag{5}$$

Therefore, the final reward is the sum of r_a and r_c.

$$r = r_a + r_c. \tag{6}$$

The balance between attractive and repulsive rewards is analogous to the APF algorithm. A larger repulsive weight ω_2 increases the repulsive forces generated by nearby obstacles, enhancing collision avoidance and ensuring the robot's safety. However, overly large ω_2 may result in excessive avoidance behavior, causing the robot to veer too far from its intended path or even prevent it from reaching the goal. Conversely, a larger attractive weight ω_1 strengthens the robot's attraction towards the target, promoting faster and more direct convergence to the goal. Yet, if ω_1 is too large relative to ω_2,the robot may fail to adequately account for obstacles, leading to a higher risk of collisions.

4 Path Replanning

4.1 Replanning Framework

To respond swiftly to the movement of obstacles, the robot must frequently sense its environment, requiring the planner to be interrupted to update the environmental description. Our method runs the path execution and verification modules in separate *Processes*, ensuring that execution is not interrupted by the verification *Process*. When the system receives the latest obstacle information, both this information and the current position of the manipulator are sent to the verification *Process*. The verification *Process*

continuously performs collision detection between the obstacle and the robot for every configuration along the current path. If a collision is detected, the planner devises a new collision-free trajectory while simultaneously starting a new collision detection cycle. If no feasible path can be found, the execution of current path will terminate. The execution *Process* checks if the robot has reached its goal and terminates the path execution if the manipulator collides with an obstacle (Fig. 2).

4.2 Replanning Process Optimisation

In dynamic environments, real-time replanning is critical to ensuring safety during the robot's operation. As shown in Fig. 3, when we are planning a trajectory (x_0, x_1, …, x_{goal}), we must get the state data through interaction with the environment. The environment is initially at state s_0, the corresponding action a_0 the agent is going to take can be predicted with the actor neural network. The agent takes that step and transfers to the new state s_1. Then a new action a_1 can be predicted according to the new state s_1 and transfers to another new state s_2. The agent repeats this cycle until the target state is reached and we can get all the states and actions. The position of the end-effector can be extracted from the states and connected to form a trajectory.

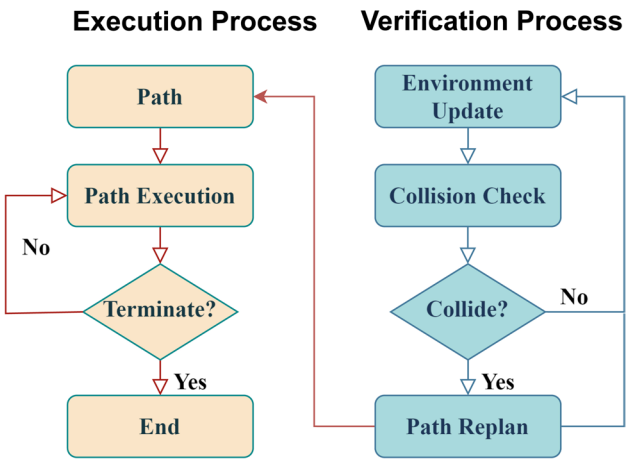


Fig. 2 Path execution and verification in separate *Processes*

We call the generation of a trajectory node a step. The planning time increases as the number of steps. Each step consists of two elements, the action prediction, and the state transition. Here we propose three improvements to speed up planning.

4.2.1 Incremental Planning

A traditional solution for replanning is to run the trajectory planning from scratch. However, if most of the existing path is collision-free, re-solving the entire planning problem duplicates work. Inspired by incremental GPMP2 [24, 25] and JIST [28], We suggest an incremental approach to update the current solution given new information. When the obstacle collides with the future state of the manipulator, instead of planning the entire trajectory as Fig. 4(b), we only remove the collision states and compute a collision-free trajectory between the two closest states x_s' and x_{goal}' with the TD3 planner as Fig. 4(c). The shortening of the replanning path brings two benefits. One is that shorter steps can reduce the difficulty of planning and increase the success rate. Another is that it can significantly reduce planning steps, thus reducing path generation time.

4.2.2 Determinstic Policy

The time required for action prediction is closely related to the type of policy used to generate actions. A deterministic policy generates actions directly from the neural network, allowing for faster and more straightforward decision-making by selecting the action with the highest predicted value. In contrast, a stochastic policy first produces a probability distribution over possible actions and then samples from this distribution to choose an action. This sampling process gives the agent better exploration capabilities during training, helping it discover more comprehensive strategies.

However, in the planning phase, where the objective is to compute a single optimal path, the benefits of a stochastic policy's exploration are less relevant. Since planning is only done once, sampling from a distribution does not necessarily lead to a better outcome and often introduces variability. Moreover, the stochastic policy requires additional

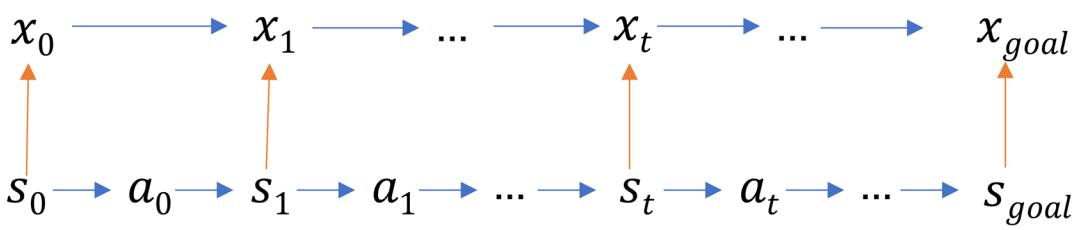


Fig. 3 Trajectory generation process

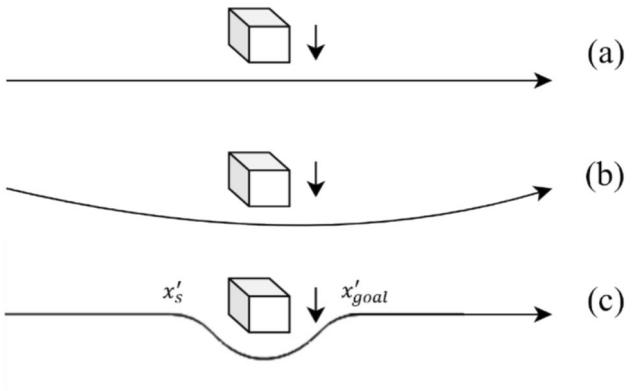


Fig. 4 Incremental path replanning

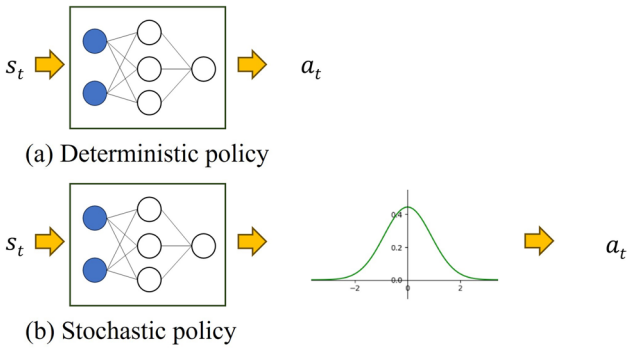


Fig. 5 Deterministic policy and stochastic policy in generating actions

computational time for action selection, which could be detrimental in real-time applications.

Therefore, the deterministic policy is typically more appropriate for real-time path planning, as it provides faster and more consistent action selection without unnecessary computational overhead. While the stochastic policy is valuable during training for enhancing exploration, its advantages diminish in the planning phase, where speed and predictability are more critical than exploration (Fig. 5).

4.2.3 Simplication of State Transition

Once a_t is obtained, the state s_t can transfer to the new state s_{t+1} through a step function as follows.

$$< s_{t+1}, r, done, info >= step(a_t), \tag{7}$$

where r is a reward for taking the action a_t, *done* indicates whether the episode terminates, and *info* contains user-defined information, such as whether the agent reaches the target position successfully. Reward r is crucial for effectively training the agent but is useless for generating actions. Here we simplify the step function in the path planning phase to remove redundant calculations, further reducing the planning time.

$$< s_{t+1}, done, info >= step(a_t). \tag{8}$$

5 Experiment and Result

5.1 Experiment

The simulation experiment was done on a computer with a CPU of i5-12400F and a GPU of RTX4070 TI SUPER. We employed the PyBullet platform in the simulation experiments. An IRB 120 robot of ABB Robotics was introduced to the environment, and we can obtain the state and execute the action using the application programming interface (API). The platform also provides the interface to detection collision (getContactPoints()) and returns the point pairs whose distances d_c are less than the influence distance d_{in} between the robot arm and the obstacle (getClosestPoints()), as required in Formula 5.

The task of the robotic arm is to get to the target position from the starting position, with obstacles in between moving at random speeds. The simulation environment is shown in Fig. 6.

As shown in Fig. 6, the original position of the manipulator is on the right side. The orientation of the end-effector is set to (0,1.57,1.57). The target object is the green cube on the left side of the workspace with a side length of 0.05 m. The obstacle is the red cube of the same size in the middle of the workspace. During a complete reaching task, the start position of the manipulator and the target object is fixed. The obstacle moves vertically from 0.8 m to 1.4 m at a random speed. When training the replanning planner, the start state and the target state are randomly selected on either side of the obstacle. Specifically, the start state is selected from the right side of the space, and the target state is selected from the left side.

In training the TD3 planner, the algorithm returns a reward for each step to guide the action of the manipulator. The episode terminates when the manipulator collides with the obstacle or reaches the target position, which returns rewards $R_t = 1000$ and $R_c = -1000$ respectively. The reward weights ω_1 and ω_2 are set to 300 and 10000. The influence distance is set to 0.02 m, only the points within this distance will be used to calculate the repulsive cost. Once the distance between the end-effector and the target object is less than 0.1 m, we assume that the manipulator has reached the target position.

The hyperparameters of the TD3 algorithm are shown in Table 2. The selection of these hyperparameters is based on both theoretical principles and empirical tuning to achieve optimal performance. The **learning rate** was chosen to

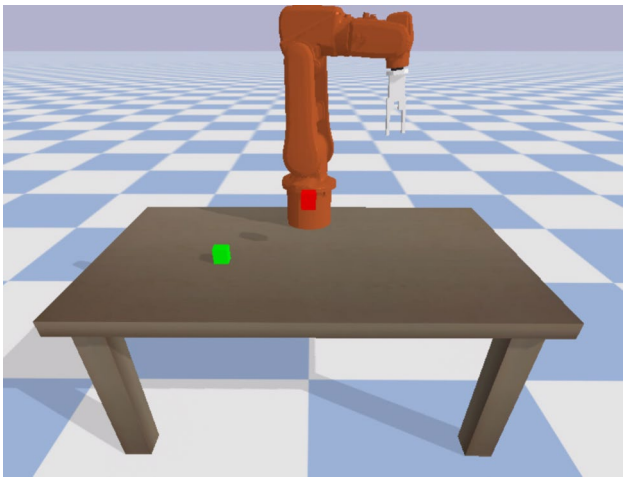


Fig. 6 Simulation environment for training

ensure stable network updates, preventing divergence while maintaining efficient learning speed. A large **replay buffer size** was selected to store a broad range of experiences, which helps in decorrelating samples and improving the generalization of the learning process. The **batch size** was adjusted to balance between efficient computation and stable gradient updates. A smaller batch size resulted in noisier updates, while larger sizes provided diminishing returns in performance. The **discount factor** was selected to emphasize long-term rewards, which is essential for complex tasks that require future-oriented planning. The **maximum episode step** was empirically determined to allow the agent enough time to complete tasks without extending training unnecessarily. The **target network update frequency** and **soft update factor** were carefully tuned to ensure that the target network evolves gradually, which is critical for maintaining training stability. Faster updates led to instability, while slower updates hindered learning progress. These hyperparameters were fine-tuned through iterative testing to optimize the balance between stability and performance for the TD3 algorithm in the specific context of robotic control tasks.

As Fig. 1 shows, the TD3 networks comprise Multilayer Perceptron (MLP) layers. The actor network consists of an input layer with 9 nodes (representing the state s_t), an output layer with 3 nodes (representing the action a_t), and two sequential hidden layers with 400 and 300 nodes, respectively. Each critic network comprises an input layer with 12 nodes (the dimension of s_t and a_t), followed by a hidden layer structure identical to that of the actor network, and an output layer to score the action. The activation functions are all Rectified Linear Units (ReLU), except for the output layer of the actor network, which uses Tanh to ensure that the output actions reside within the continuous action space and are bound within a limited range, which is determined by the physical constraints of the robot's joints. Target networks mirror the corresponding network structures to facilitate training stability.

Table 2 Hyperparameters of TD3

Parameter	Value
Learning rate	0.001
Replay buffer size	10^6
Batch size	128
Discount factor	0.99
Episode maximum step	50
Target network update frequency (time steps)	1
Soft update factor	3e-3

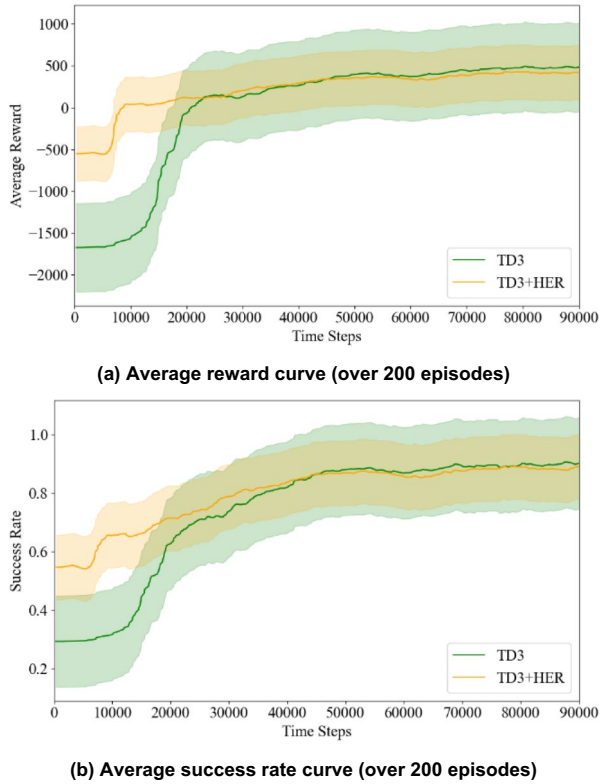
5.2 Result

5.2.1 Validation of TD3 Planner

As shown in Fig. 7, the agent can learn how to generate a new collision-free trajectory with the setting of the environment, including the reward function and the setting of the state space and the action space. The training reward increases quickly and soon converges to get the best policy. When the timestep reaches about 70,000, the success rate reaches about 95%.

To evaluate the impact of HER on training performance, experiments using TD3 combined with HER were conducted. The results demonstrate that HER can significantly reduce the training time required to achieve a relatively high success rate during the early stages of training. However, the final success rate of TD3 with HER stabilizes around 90%, which is approximately 5% lower than that of the standard TD3. This indicates that HER adversely affects the ultimate planning success rate. A similar trend is observed with SAC and DDPG, as illustrated in Fig. 8. In the context of path replanning, achieving a higher success rate is more crucial than minimizing training time. Additionally, the pure TD3 algorithm demonstrates sufficiently fast learning speed given the environmental settings.

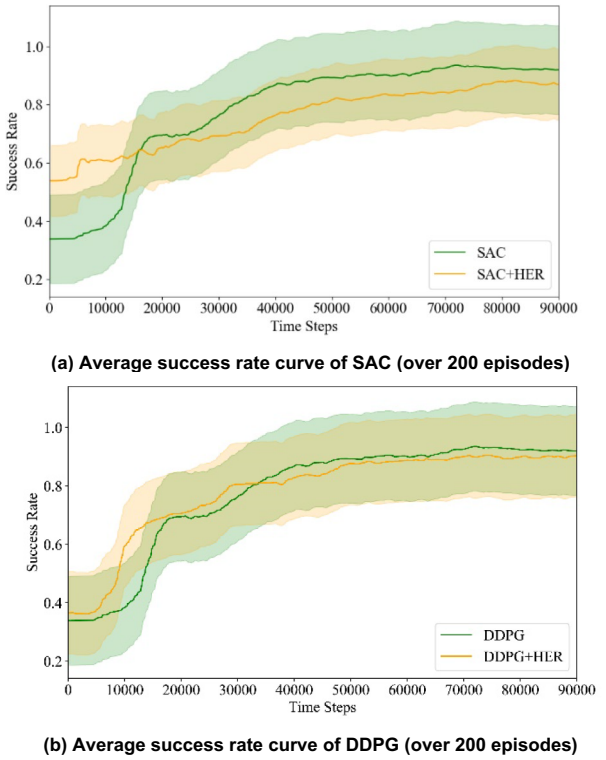
Throughout the training process, TD3 shows negligible differences compared to DDPG and SAC, as shown in Fig. 9. DDPG produces a superior policy in the early stages but converges more slowly than the others. SAC converges slightly faster initially but takes a comparable amount of time to reach a stable result as TD3. The final success rate of all three algorithms—TD3, SAC, and DDPG—approaches 95%, with peak success rates reaching 98%.



(a) Average reward curve (over 200 episodes)

(b) Average success rate curve (over 200 episodes)

Fig. 7 The training process of the reaching task. (**a**) Average reward curve (over 200 episodes). (**b**) Average success rate curve (over 200 episodes)



(a) Average success rate curve of SAC (over 200 episodes)

(b) Average success rate curve of DDPG (over 200 episodes)

Fig. 8 The influence of HER on success rate. (**a**) Average success rate curve of SAC (over 200 episodes). (**b**) Average success rate curve of DDPG (over 200 episodes)

5.2.2 Planning Efficiency Improvement

Suppose the obstacle moves when the manipulator approaches the obstacle, the planner needs to replan the trajectory to avoid potential collision. The incremental planner keeps most of the trajectory unaffected, only sending a new planning request to connect the nearest two states. In contrast, the ordinary planner generates an entirely new path from the current position to the target.

We conducted a comparison between these two approaches, running the replanning process 1000 times for each algorithm. The resulting trajectories are illustrated in Fig. 10, and the total planning times are shown in Fig. 11.

The ITD3 column displays the replanning time for the incremental TD3 planner. ITD3-S (incremental TD3 Simplified) represents the planning time after simplifying the state transition function. ITD3-H (incremental TD3 with HER) shows the planning time after integrating Hindsight Experience Replay (HER). TD3 represents the time required to replan the entire trajectory from the current state of the manipulator using the standard TD3 algorithm. The naming convention for other items follows a similar pattern.

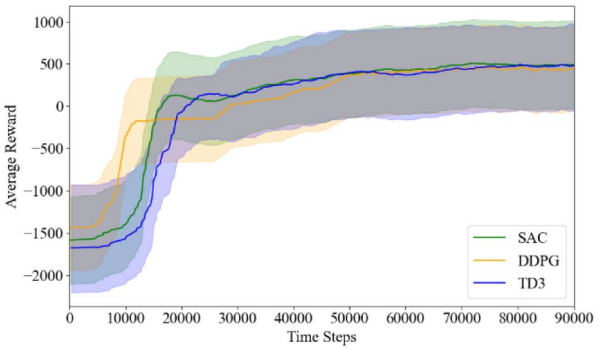
In terms of baseline algorithms, the planning times of DDPG and TD3 are akin due to their deterministic nature, boasting a speed roughly 8.9% $\left(1-\frac{14.4}{15.8}\right)$ faster than SAC. However, the integration of HER introduces a slight increase in planning time across all algorithms, approximately 3% in total.

Notably, the proposed incremental strategy, which only replanning the obstructed portion of the trajectory, saves about 6.9% $\left(1-\frac{13.4}{14.4}\right)$ of the planning time compared to replanning the entire trajectory. The simplification of the step function reduces the planning time by 10%.

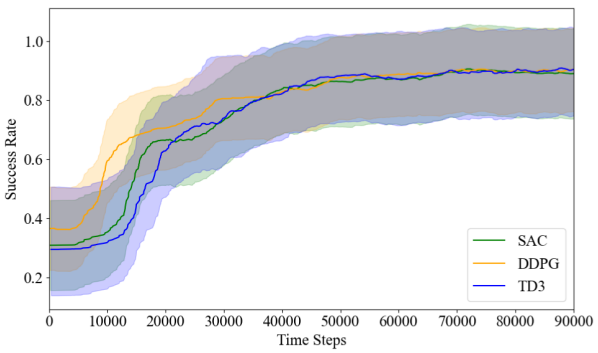
The proposed approach, combining the simplified step function and partial replanning, reduces planning time by approximately 16.0% $\left(1-\frac{12.1}{14.4}\right)$ compared to replanning the entire trajectory with vanilla TD3, and 23.4% $\left(1-\frac{12.1}{15.8}\right)$ compared to vanilla SAC.

5.2.3 Performance Comparisons

For each algorithm, the described planning task was executed 1000 times, with obstacle positions being randomly reset for each iteration. The experimental results are presented in Table 3. The details of how the comparison algorithms were implemented are outlined in Appendix A for



(a) Average reward curve (over 200 episodes)



(b) Average success rate curve (over 200 episodes)

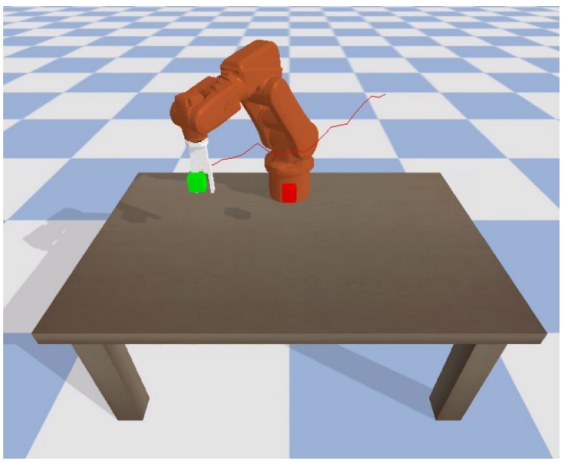
Fig. 9 Comparison between SAC, DDPG, and TD3. (**a**) Average reward curve (over 200 episodes). (**b**) Average success rate curve (over 200 episodes)

further reference. Smoothness and roughness are calculated according to Eqs. (12) and (13), respectively.

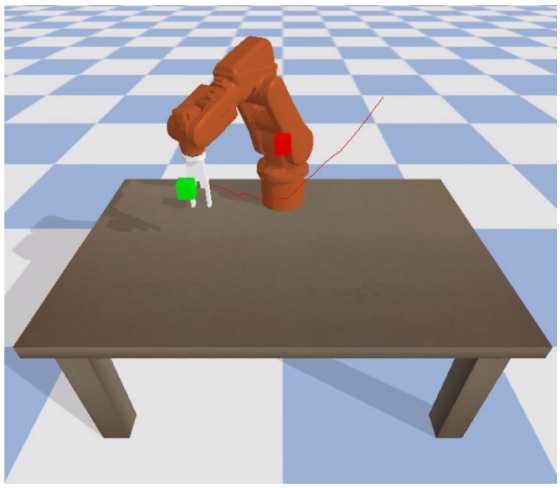
Among Sampling-based algorithms, RRT-connect showed the fastest planning speed and generated the shortest paths. While the performance is related to the complexity of the planning problem, a dense obstacle environment may limit its performance.

Optimization-based algorithms typically demonstrate shorter planning times compared to sampling-based algorithms [28, 62], provided that the SDF utilized for collision detection is precomputed. By precomputing the SDF, collision detection is expedited through rapid lookups, thereby significantly increasing the overall efficiency of the optimization process.

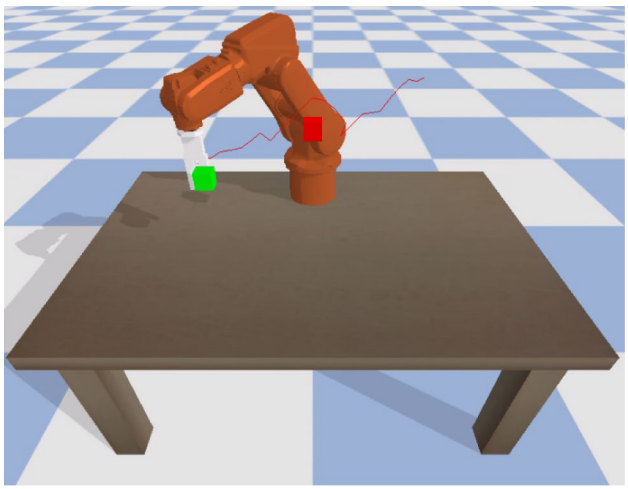
In our experimental environment, the collision cost function relies on the 'getClosestPoints' API call for each calculation. This API call is significantly more time-consuming (0.0068 ms per call) compared to simple numerical computations. Optimization-based algorithms require iterative path refinement, and each iteration necessitates re-evaluating the collision status for points along the path. Consequently, this repeated invocation of 'getClosestPoints' results in higher computational times for optimization-based algorithms in this setup.



(a) Original path



(b) Entirely replanned path



(c) Partially replanned path

Fig. 10 Replanned paths with different methods. (**a**) Original path. (**b**) Entirely replanned path. (**c**) Partially replanned path

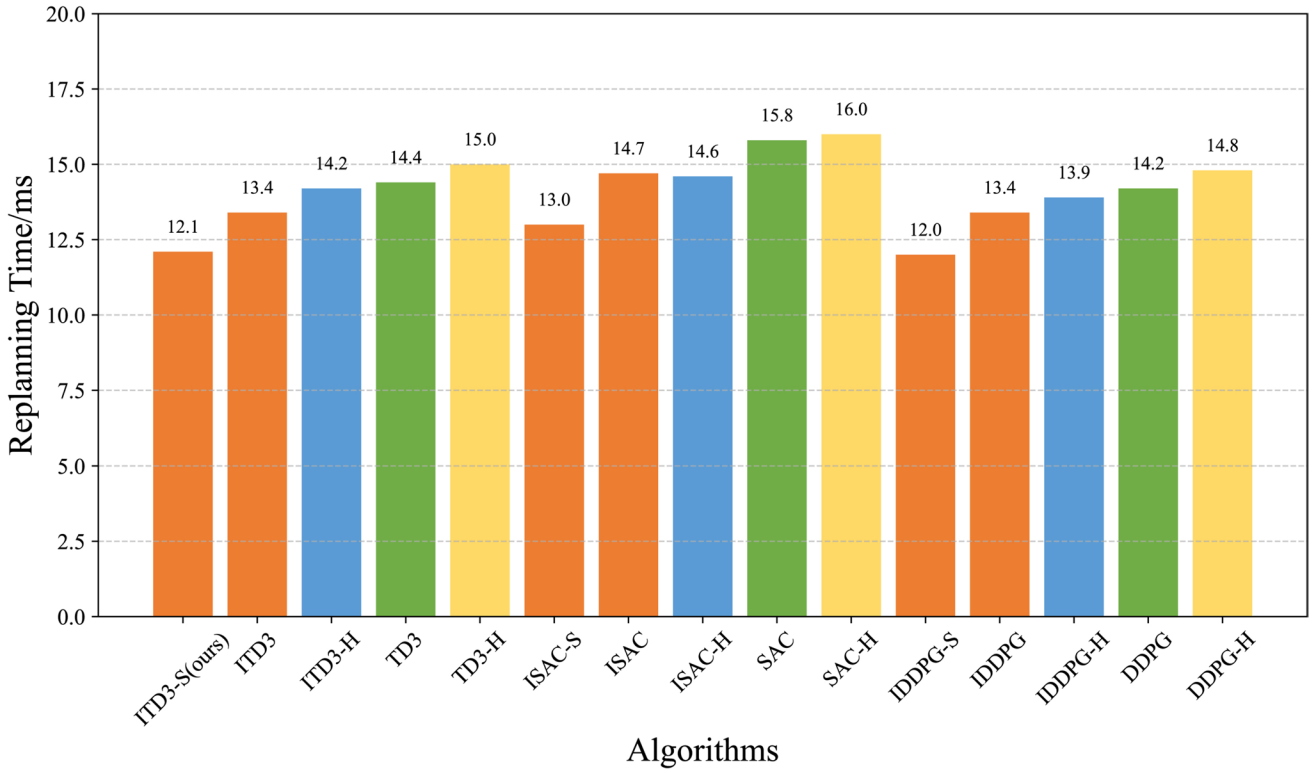


Fig. 11 Performance comparisons of our method with the vanilla DRL-based method

Table 3 Performance comparisons of our method with Popular Algorithms

Method	Type	Time(s)	Path length(m)	Roughness	Smoothness
RRT[4]	Sampling-based Algorithms	4.53E-02	1.161	0.0986	0.959
Bias-goal RRT[6]		3.83E-03	0.935	0.0712	0.664
PRM[61]		1.41E + 00	0.869	0.0496	0.589
RRT*[63]		5.93E-01	0.786	0.1807	0.587
RRT-connect[64]		1.02E-03	0.758	0.0086	0.112
BIT*[65]		3.77E-01	1.045	0.0768	0.892
CHOMP[21]	Optimization-based Algorithms	3.16E-01	0.759	0.0018	0.067
STOMP[22]		4.03E + 00	0.740	0.0147	0.280
TrajOpt[66]		3.71E-01	0.760	0.0025	0.084
DDPG[35]	RL-based Algorithms	1.42E-02	0.759	0.0046	0.167
TD3[41]		1.44E-02	0.759	0.0024	0.134
SAC[67]		1.58E-02	0.769	0.0062	0.269
ITD3-S(ours)		1.21E-02	0.772	0.0042	0.187

Optimization-based algorithms achieved near-optimal levels in terms of path length, roughness, and smoothness, demonstrating their advantages in planning high-quality paths. CHOMP, based on Covariant Gradient Descent, converges faster and produces better results compared to methods that approximate the gradient using finite differences.

RL-based Algorithms achieve path lengths and roughness comparable to optimization-based algorithms in just one-third $\left(\frac{1.44\text{E}-02}{4.53\text{E}-02} = 31.8\%\right)$ of the time taken by RRT.

Our method reduced planning time by approximately $16.0\%\left(1-\frac{12.1}{14.4}\right)$ compared to replanning the entire trajectory with vanilla TD3, combined the simplified step function and partial replanning strategy.

6 Conclusion

In response to the safety issues of path planning methods based on deep reinforcement learning (DRL), this article proposes integrating a safety verification mechanism into the DRL-based planner. This system predicts potential collisions and generates alternative collision-free paths as necessary, ensuring a collision-free trajectory before execution.

We analyzed the necessary elements in the planning process and formulated methods to reduce the time required for each element. Firstly, the planning time increases with the number of planning steps. To address this, we propose an incremental path-planning method. When an obstacle is detected, only the affected part of the path is modified to compute a new trajectory, thereby reducing the number of planning steps by connecting the unaffected parts of the original path. For action prediction, we employ a deterministic policy method, specifically TD3, the state-of-the-art deterministic algorithm. Unlike stochastic methods, the deterministic algorithm outputs actions directly without a sampling process. With a finely designed reward function, the success rate of the TD3 path planner reaches 95%. To optimize state transitions, we remove redundant calculations in the transition function, reducing transition time. Simulation experiments demonstrate that the proposed method reduces calculation time by approximately 16.0% compared to the vanilla TD3 planner and 23.4% compared to a SAC planner. The average planning time of the proposed algorithm is 12.1 ms, significantly faster than RRT, enabling timely responses to obstacles. Furthermore, the path generated is nearly optimal as Optimization-based Algorithms.

However, the proposed method is not without its limitations. The structure of the state space causes the fixation of the end-effector orientation, necessitating inverse kinematics calculations in the step function, which impairs the manipulator's flexibility and increases state transition time. Additionally, assuming the manipulator reaches the target position when the distance is below a threshold result in a double threshold distance when two trajectories are joined. This requires a smaller threshold, which increases the number of planning steps.

In future work, we intend to utilize visual sensors to dynamically perceive obstacle positions. We will then integrate our proposed control algorithm into practical robotic controllers to validate its feasibility in real robotic systems. By incorporating visual perception, we aim to enable real-time environmental awareness and adaptability for robots, enhancing their motion planning and control capabilities in complex and dynamic environments.

Appendix A

Implementation Details

A. Sampling-Based Algorithms

In sampling-based algorithms, the orientation of the end-effector remains constant throughout the planning process. The planning is performed by first sampling positions for the Tool Center Point (TCP) within the workspace. Then, inverse kinematics is applied to compute the joint angles for these positions. To avoid collisions, the 'getContactPoints' API in PyBullet is used for collision detection.

1) Common Parameters for Sampling-Based Algorithms
2) Max Samples: The algorithm can generate up to 1000 random samples to grow the tree.
3) Step Length: The maximum step length for tree extension towards a sampled point is 0.1 m per iteration.
4) Collision Checking: Performed every 0.02 m when connecting path points to ensure no collisions.
5) Goal Threshold: The goal is considered reached if the tree reaches within 0.1 m of the goal position.
6) Specific Algorithm Parameters
7) Bias Goal RRT: A bias factor of 0.5 is used, meaning that after determining the nearest node to a random sample, an additional 50% of the maximum step length is taken towards the goal.
8) RRT*: A search radius of 0.5 m is applied for local optimization to minimize path length.
9) PRM: A sample size of 1000 is employed with a maximum step length of 0.1 m for smooth transitions, and 10 nearest neighbors are considered to improve connectivity.
10) BIT*: Similar to PRM, a sample size of 1000 and a maximum step length of 0.1 m are used. Heuristic and cost calculations are also used to reduce path length.

B. Optimization-Based Algorithms

In optimization-based algorithms, the path planning problem is framed as an optimization task. The objective is to find a feasible solution by minimizing a cost function while meeting certain constraints. The general formulation for optimization-based motion planning can be expressed as:

$$\begin{aligned} &\text{minimize} && \mathcal{F}[\theta(t)] \\ &\text{subject to} && \mathcal{G}_i[\theta(t)] \le 0, i = 1, \ldots, m_{ineq} \\ &&& \mathcal{H}_i[\theta(t)] = 0, i = 1, \ldots, m_{eq} \end{aligned} \tag{9}$$

1) Common Setting for Optimization-Based Algorithms

The objective function consists of multiple cost components, including path length, obstacle avoidance, smoothness, and roughness. The function $\mathcal{F}$ is expressed as:

$$\mathcal{F} = R_l + R_\mathrm{s} + R_\mathrm{r} + R_\mathrm{o} \tag{10}$$

where: R_l is the path length cost:

$$R_l = \sum_{t=1}^{T} \| \frac{ds_t}{dt} \| \tag{11}$$

R_s is the smoothness cost:

$$R_\mathrm{s} = \sum_{t=1}^{T} \| \frac{d^2 s_t}{dt^2} \| \tag{12}$$

R_r is the roughness cost [100]:

$$R_r = \sum_{t=1}^{T} \left(\frac{d^2 s_t}{dt^2} \right)^2 \tag{13}$$

R_o is the obstacle cost:

$$R_o = \sum_{t=1}^{T} \alpha \sum_{i} \left(\beta - d_{t,i} \right)^2 \tag{14}$$

Here, S_t represents the position of the TCP at the t-th discrete point, and $T = 10$ is the total number of discrete points. $d_{t,i}$ is the distance between the robot and the i-th closest obstacle at the t-th waypoint, as determined by the closest points, which are obtained by the API 'getClosestPoints'. $\alpha = 100$ is a scaling factor for the obstacle cost. $\beta = 0.1$ meter is the threshold distance below which the obstacle cost starts to increase.

- Inequality Constraints: The joint angles $\theta_j(t)$ must stay within allowable limits:

$$\theta_j^{min} \leq \theta_j(t) \leq \theta_j^{max}, j = 1, \ldots, n \tag{15}$$

where n is the number of joints.

- Equality constraints: Ensure that the initial and final positions are at the start and goal positions:

$$s\left(t_0\right) = s_\mathrm{start}, s(t_f) = s_\mathrm{goal}. \tag{16}$$

- A straight-line trajectory is generated as an initial guess, and optimization refines the trajectory until either a convergence threshold of $1\mathrm{e}-4$) is reached or the maximum of 100 iterations is completed.
- Specific Algorithm Parameters:
- TrajOpt: Gradients are computed by perturbing trajectory points by a small value (1e-5) and observing the changes in cost. The trajectory points are iteratively updated using a learning rate of 0.01 until convergence or the maximum number of iterations.
- CHOMP: Like TrajOpt, CHOMP computes gradients through perturbation (epsilon = 1e-5) but updates points using Covariant Gradient Descent. A Hessian approximation is used to speed up the process.
- STOMP: This algorithm uses a stochastic method, generating noisy trajectories with random perturbations (standard deviation of 0.01) and then updating points based on the cost of these trajectories, iterating until convergence or the iteration limit.

Author Contributions Conceptualization: [JL], [HJY], [ASMK]; Methodology: [JL], [HJY], [ASMK]; Formal analysis and investigation: [JL]; Writing—original draft preparation: [JL]; Writing—review and editing: [HJY], [ASMK]; Funding acquisition, resources: [HJY], [JL]; Supervision: [HJY], [ASMK].

Funding This work was supported by The Ministry of Higher Education for the Fundamental Research Grant Scheme (FRGS/1/2022/TK10/UM/02/7) awarded to Ir. Dr. Hwa-Jen Yap (Universiti Malaya) and Application Innovation Project of Hebei Vocational University of Technology and Engineering (202205).

Data Availability Data will be made available upon reasonable request.

Declarations

Competing Interests The authors have no competing interests to disclose about the content of this article.

References

1. Zabalza, J., Fei, Z., Wong, C., Yan, Y., Mineo, C., Yang, E., Rodden, T., Mehnen, J., Pham, Q., Ren, J.: Smart sensing and adaptive reasoning for enabling industrial robots with interactive human-robot capabilities in dynamic environments: a case study. Sensors **19**(6), 1354 (2019)
2. Nicola, G., Ghidoni, S.: Deep Reinforcement Learning for Motion Planning in Human Robot cooperative Scenarios. in 2021 26th

IEEE International Conference on Emerging Technologies and Factory Automation (ETFA). IEEE, 1–7 (2021).
3. Li, S., Han, K., Li, X., Zhang, S., Xiong, Y., Xie, Z.: Hybrid trajectory replanning-based dynamic obstacle avoidance for physical human-robot interaction. J. Intell. Rob. Syst. **103**(3), 1–14 (2021)
4. LaValle, S.: Rapidly-exploring random trees: a new tool for path planning. Res. Rep. 9811 (1998).
5. Long, H., Li, G., Zhou, F., Chen, T.: Cooperative dynamic motion planning for dual manipulator arms based on RRT*Smart-AD algorithm. Sensors **23**(18), 7759 (2023)
6. Yuan, C., Shuai, C., Zhang, W.: A dynamic multiple-query RRT planning algorithm for manipulator obstacle avoidance. Appl. Sci. Basel **13**(6), 3394 (2023)
7. Yu, Y., Zhang, Y.: Collision avoidance and path planning for industrial manipulator using slice-based heuristic fast marching tree. Robot. Comput.-Integr. Manuf **75**, 102289 (2022)
8. Merckaert, K., Convens, B., Nicotra, M., Vanderborght, B.: Real-time constraint-based planning and control of robotic manipulators for safe human-robot collaboration. Robot. Comput.-Integr. Manuf **87**, 102711 (2024)
9. Wei, S., Liu, B., Yao, M., Yu, X., Tang, L.: Efficient online motion planning method for the robotic arm to pick-up moving objects smoothly with temporal constraints. Proc. Inst. Mech. Eng **236**(15), 8650–8662 (2022)
10. Dam, T., Chalvatzaki, G., Peters, J., Pajarinen, J.: Monte-Carlo robot path planning. IEEE Robot. Autom. Lett **7**(4), 11213–11220 (2022)
11. Cao, X., Zou, X., Jia, C., Chen, M., Zeng, Z.: RRT-based path planning for an intelligent litchi-picking manipulator. Comput. Electron. Agric. **156**, 105–118 (2019)
12. Yuan, C., Liu, G., Zhang, W., Pan, X.: An efficient RRT cache method in dynamic environments for path planning. Robot. Auton. Syst. **131**, 103595 (2020)
13. Zhang, H., Wang, Y., Zheng, J., Yu, J.: Path planning of industrial robot based on improved RRT algorithm in complex environments. IEEE Access **6**, 53296–53306 (2018)
14. Ichter, B., Harrison, J., Pavone, M.: Learning sampling distributions for robot motion planning. in 2018 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 7087–7094 (2018).
15. Wang, J., Chi, W., Li, C., Wang, C., Meng, M.: Neural RRT*: learning-based optimal path planning. IEEE Trans. Autom. Sci. Eng. **17**(4), 1748–1758 (2020)
16. Ma, N., Wang, J., Liu, J., Meng, M.: Conditional generative adversarial networks for optimal path planning. IEEE Trans. Cogn. Dev. Syst. **14**(2), 662–671 (2022)
17. Wang, Y., Wei, L., Du, K., Liu, G., Yang, Q., Wei, Y., Fang, Q.: An online collision-free trajectory generation algorithm for human-robot collaboration. Robot. Comput.-Integr. Manuf **80**, 102475 (2023)
18. Power, T., Berenson, D.: Learning a generalizable trajectory sampling distribution for model predictive control. IEEE Trans. Rob. **40**, 2111–2127 (2024)
19. Lee, C., Song, K.: Path re-planning design of a cobot in a dynamic environment based on current obstacle configuration. Robot. Autom. Lett. **8**(3), 1183–1190 (2023)
20. Jiang, L., Liu, S., Cui, Y., Jiang, H.: Path planning for robotic manipulator in complex multi-obstacle environment based on improved_RRT. IEEE/ASME Trans. Mechatron. **27**(6), 4774–4785 (2022)
21. Ratliff, N., Zucker, M., Bagnell, J., Srinivasa, S.: CHOMP: Gradient optimization techniques for efficient motion planning. In: IEEE International Conference on Robotics and Automation (ICRA). IEEE, 489–494 (2009).
22. Kalakrishnan, M., Chitta, S., Theodorou, E., Pastor, P., Schaal, S.: STOMP: Stochastic trajectory optimization for motion planning. In: IEEE International Conference on Robotics and Automation (ICRA). IEEE, 4569–4574 (2011).
23. Park, C., Pan, J., Manocha, D.: ITOMP: Incremental Trajectory Optimization for Real-time Replanning in Dynamic Environments. In: IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 22, 207–215 (2012).
24. Finean, M., Petrovic, L., Merkt, W., Markovic, I., Havoutis, I.: Motion planning in dynamic environments using context-aware human trajectory prediction. Robot. Auton. Syst. **166**, 104450 (2023)
25. Dong, J., Mukadam, M., Dellaert, F., Boots, B.: Motion Planning as Probabilistic Inference using Gaussian Processes and Factor Graphs. In: Robotics: Science and Systems (RSS). 12(4), (2016).
26. Finean, M., Merkt, W., Havoutis, I.: Simultaneous Scene Reconstruction and Whole-Body Motion Planning for Safe Operation in Dynamic Environments. In: IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 3710–3717 (2021)..
27. Kuntz, A., Bowen, C., Alterovitz, R.: Fast Anytime Motion Planning in Point Clouds by Interleaving Sampling and Interior Point Optimization. In: Springer International Conference on Intelligent Robots and Systems (IROS). Springer, 929–945 (2020).
28. Alwala, K., Mukadam, M.: Joint Sampling and Trajectory Optimization over Graphs for Online Motion Planning. In: IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 4700–4707 (2021).
29. Watkins, C. J. C. H.: Learning from delayed rewards. PhD Thesis, King's College, University of Cambridge (1989)
30. Salmaninejad, M., Zilles, S., Mayorga, R.: Motion Path Planning of Two Robot Arms in a Common Workspace. In: IEEE International Conference on Systems, Man, and Cybernetics (SMC). IEEE, 45–51 (2020).
31. Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., Riedmiller, M.: Playing Atari with Deep Reinforcement Learning. arXiv:1312.5602 (2013).
32. Petrenko, V., Tebueva, F., Ryabtsev, S., Gurchinsky, M.: Method of Controlling the Movement of an Anthropomorphic Manipulator in the Working Area With Dynamic Obstacle. In: 8th Scientific Conference on Information Technologies for Intelligent Decision Making Support (ITIDS). IEEE, 359–364 (2020).
33. Alam, M. S., Sudha, S. K. R., Somayajula, A.: AI on the Water: Applying DRL to Autonomous Vessel Navigation. arXiv preprint arXiv:2310.14938 (2023).
34. Regunathan, R.D., Sudha, S.K.R., Alam, M.S., Somayajula, A.: Deep Reinforcement Learning Based Controller for Ship Navigation. Ocean Eng. **273**, 113937 (2023)
35. Lillicrap, T., Hunt, J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., Wierstra, D.: Continuous control with deep reinforcement learning. arXiv:1509.02971 (2015).
36. Li, Z., Ma, H., Ding, Y., Wang, C., Jin, Y.: Motion planning of six-dof arm robot based on improved DDPG algorithm. In: 2020 39th Chinese Control Conference (CCC). IEEE, 3954–3959 (2020).
37. Lindner, T., Milecki, A.: Reinforcement learning-based algorithm to avoid obstacles by the anthropomorphic robotic arm. Appl. Sci. **12**, 6629 (2022)
38. Zeng, R., Liu, M., Zhang, J., Li, X., Zhou, Q., Jiang, Y.: Manipulator Control Method Based on Deep Reinforcement Learning. In: 2020 Chinese Control And Decision Conference (CCDC). IEEE, 415–420 (2020).
39. Um, D., Nethala, P., Shin, H.: Hierarchical DDPG for manipulator motion planning in dynamic environments. AI **3**(3), 645–658 (2022)
40. Jose, J., Alam, M. S., Somayajula, A. S.: Navigating the Ocean with DRL: Path following for marine vessels. arXiv preprint arXiv:2310.14932 (2023).
41. Fujimoto, S., van Hoof, H., Meger, D.: Addressing function approximation error in actor-critic methods. arXiv:1802.09477 (2018).

42. Wang, S., Yi, W., He, Z., Xu, J., Yang, L.: Safe reinforcement learning-based trajectory planning for industrial robot. In: IEEE International Conference on Systems, Man, and Cybernetics (SMC). IEEE, 3471–3476 (2020).
43. Huang, Z., Chen, G., Shen, Y., Wang, R., Liu, C., Zhang, L.: An obstacle-avoidance motion planning method for redundant space robot via reinforcement learning. Actuators **12**(2), 69 (2023)
44. Chen, P., Pei, J., Lu, W., Li, M.: A deep reinforcement learning based method for real-time path planning and dynamic obstacle avoidance. Neurocomputing **497**, 64–75 (2022)
45. Schaul, T., Quan, J., Antonoglou, I., Silver, D.: Prioritized Experience Replay. CoRR abs/1511.05952 (2015).
46. Andrychowicz, M., Crow, D., Ray, A., Schneider, J., Fong, R., Welinder, P., ..., Zaremba, W.: Hindsight Experience Replay. ArXiv abs/1707.01495 (2017).
47. Feng, X.: Consistent experience replay in high-dimensional continuous control with decayed hindsights. Machines **10**, 856 (2022)
48. Kim, S., An, B.: Learning Heuristic A: Efficient Graph Search using Neural Network. In: 2020 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 9542–9547 (2020).
49. Prianto, E., Park, J.H., Bae, J.H., Kim, J.S.: Deep reinforcement learning-based path planning for multi-arm manipulators with periodically moving obstacles. Applied Sciences-Basel **11**(6), 2587 (2021)
50. Ren, Z., Dong, K., Zhou, Y., Liu, Q., Peng, J.: Exploration via Hindsight Goal Generation. Adv. Neural Inf. Process Syst. 32 (2019).
51. Bing, Z., Brucker, M., Morin, F.O., Li, R., Su, X., Huang, K., Knoll, A.: Complex robotic manipulation via graph-based hindsight goal generation. IEEE Trans. Neural Netw. Learn. Syst **33**(12), 7863–7876 (2021)
52. Bing, Z. S., Alvarez, E., Cheng, L., Morin, F. O., Li, R., Su, X. J., ..., Knoll, A.: Robotic Manipulation in Dynamic Scenarios via Bounding-Box-Based Hindsight Goal Generation. IEEE Trans. Neural Netw. Learn. Syst. 34(8), 5037–5050 (2023).
53. Althoff, M., Dolan, J.M.: Online verification of automated road vehicles using reachability analysis. IEEE Trans. Rob. **30**(4), 903–918 (2014)
54. Chan, C.C., Tsai, C.C.: Collision-free path planning based on new navigation function for an industrial robotic manipulator in human-robot coexistence environments. J. Chin. Inst. Eng. **43**(6), 508–518 (2020)
55. Zhao, J. B., Zhao, Q., Wang, J. Z., Zhang, X., Wang, Y. L.: Path Planning and Evaluation for Obstacle Avoidance of Manipulator Based on Improved Artificial Potential Field and Danger Field. In: 33rd Chinese Control and Decision Conference (CCDC). IEEE, 3018–3025 (2021).
56. Tulbure, A., Khatib, O.: Closing the Loop: Real-Time Perception and Control for Robust Collision Avoidance with Occluded Obstacles. In: IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 5700–5707 (2020).
57. Zhao, M., Lv, X.Q.: Improved manipulator obstacle avoidance path planning based on potential field method. J. Robot. **2020**, 1–12 (2020)
58. Zhang, H., Zhu, Y.F., Liu, X.F., Xu, X.R.: Analysis of obstacle avoidance strategy for dual-arm robot based on speed field with improved artificial potential field algorithm. Electronics **10**(15), 1850 (2021)
59. Elahres, M., Fonte, A., Poisson, G.: Evaluation of an artificial potential field method in collision-free path planning for a robot manipulator. In: 2nd International Conference on Robotics, Computer Vision and Intelligent Systems (ROBOVIS). 92–102 (2021).
60. Khatib, O.: Real-time obstacle avoidance for manipulators and mobile robots. In: Proceedings of the 1985 IEEE International Conference on Robotics and Automation. IEEE, 500–505 (1985).
61. Kavraki, L.E., Svestka, P., Latombe, J., Overmars, M.H.: Probabilistic roadmaps for path planning in high-dimensional configuration spaces. IEEE Trans. Robot. Autom. **12**(4), 566–580 (1996)
62. Karaman, S., Frazzoli, E.: Sampling-based algorithms for optimal motion planning. Int.l J. Robot. Res. **30**(7), 846–894 (2011)
63. Mukadam, M., Dong, J., Yan, X., Dellaert, F., Boots, B.: Continuous- time Gaussian process motion planning via probabilistic inference. Int. J. Robot. Res. **37**(11), 1319–1340 (2018)
64. Thakar, S., Rajendran, P., Kim, H., Kabir, A. M., Gupta, S. K.: Accelerating bi-directional sampling-based search for motion planning of non-holonomic mobile manipulators. In: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, 6711–6717 (2020).
65. Gammell, J. D., Srinivasa, S. S., Barfoot, T. D., Batch Informed Trees (BIT): Sampling-based optimal planning via the heuristically guided search of implicit random geometric graphs. In: 2015 IEEE international conference on robotics and automation (ICRA), 3067–3074 (2015).
66. Schulman, J., Ho, J., Lee, A.X., Awwal, I., Bradlow, H., Abbeel, P.: Finding locally optimal, collision-free trajectories with sequential convex optimization. Robot Sci Syst IX **9**(1), 1–10 (2013)
67. Haarnoja, T., Zhou, A., Abbeel, P., Levine, S.: Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: International conference on machine learning (PMLR), 1861–1870 (2018).

Jie Liu received the B.S. degree in Material Processing and Control Engineering and the M.S. degree in Materials Processing Engineering from Tianjin University, Tianjin, China. He is currently pursuing the Ph.D. degree in Mechanical Engineering at the Department of Mechanical Engineering, Universiti Malaya, Kuala Lumpur, Malaysia. His research interests include motion planning, machine vision, and collaborative robots.

Ir. Dr. Hwa-Jen Yap is a Professor at the Department of Mechanical Engineering, Faculty of Engineering, Universiti Malaya, Malaysia. He obtained his B.Eng. in Mechanical Engineering, Master of Engineering Science, and Ph.D. in Engineering from Universiti Malaya. He has published more than 100 papers in reputable journals and conference proceedings and has supervised over 15 Ph.D. and Master's students. His research interests include Extended Reality (Virtual Reality, Augmented Reality, Mixed Reality), Virtual Manufacturing, Product Design, Artificial Intelligence, Sports Engineering, Industry 4.0, Robotics (Industrial and Collaborative Robots), and Automation. Dr. Yap has been involved in over 15 research & technology consultancy projects, especially related to Aerospace Manufacturing. He is also the Director of the Consultancy Centre and UPUM Sdn Bhd at Universiti Malaya.

Dr. Anis Salwa Mohd Khairuddin received her B.Sc. in Electrical Engineering from Universiti Tenaga Nasional, Malaysia, in 2008, her Master's degree in Computer Engineering from the Royal Melbourne Institute of Technology (RMIT), Australia, in 2010, and her Ph.D. in Electrical Engineering from Universiti Teknologi Malaysia in 2014. She is currently an Associate Professor at the Department of Electrical Engineering, Faculty of Engineering, Universiti Malaya, Malaysia. Her research interests include expert systems (machine learning and deep learning), agricultural robotics and automation (classification, control systems, and intelligent systems), and image and pattern recognition.